

Project's Github repo link:

<https://github.com/DanielJ0131/interactive-house>

Scope

This sprint the SG4 Innovation subgroup will support verification of the web application developed for the Interactive House project, with focus on the device-control flow used by the Devices subgroup. Testing will cover the current web interface features that are already implemented: login, signup, guest access, hub state display, device toggles, live Firestore synchronization, basic navigation and speech testing.

Purpose

The purpose of this test plan is to verify that the web application behaves correctly when interacting with Firebase Authentication and Firestore, and that device-related actions in the interface are reflected in the backend state in a reliable way.

Test Items

The following parts of the web application are included in the current sprint testing:

- Landing page navigation
- Login page
- Signup page
- Guest hub page
- Hub page
- Device toggle functions for white light, orange light, door, window, fan INA, and fan INB
- Firestore snapshot-based state updates
- Basic error handling for invalid login/signup input
- Speech

Test Approach

Testing in this sprint will mainly be manual integration testing. The main user flows will be executed through the web interface while verifying that Firebase Authentication and Firestore update as expected. If time permits, lightweight unit testing will also be explored for isolated frontend logic such as validation and error-message mapping.

Automated Test Approach

In addition to the planned manual integration testing, an initial automated test setup was added for the web application using Jest and React Testing Library. The purpose of this automated testing is to verify important frontend flows in a repeatable way without depending on the real Firebase backend during test execution.

The automated tests currently use mocked Firebase Authentication and Firestore functions. This allows the login, signup, and Firebase configuration logic to be tested safely without creating real users or writing test data to the production database.

The automated test suite currently covers:

- Login page rendering
- Login validation for empty email and empty password
- Successful login flow with mocked Firebase Authentication
- Failed login flow with a user-friendly error message
- Signup page rendering
- Signup validation for missing name, missing email, short password, and mismatched passwords
- Successful signup flow with mocked account creation, profile update, Firestore user document creation, and redirect to hub
- Failed signup flow for already registered email
- Firebase Auth configuration export
- Firebase Firestore configuration export

The automated test suite currently contains 14 passing tests across 4 test suites. Code coverage is generated using Istanbul through Jest, and the HTML coverage report can be viewed through the generated coverage/lcov-report/index.html file.

Planned Test Activities

1. Verify login flow with valid and invalid credentials
2. Verify signup flow with valid and invalid input
3. Verify guest access navigation
4. Verify hub loads device and telemetry values from Firestore
5. Verify device toggle actions update Firestore correctly
6. Verify live updates are reflected in the hub while the page is open
7. Record defects and retest after fixes

Final Automated Web Testing Status

The web application was tested using Jest, React Testing Library, and mocked Firebase services. The final automated test suite contains 49 passing tests across 9 test suites.

The tests cover login, signup, Firebase Auth configuration, Firestore configuration, shared UI components, static pages, guest hub, emergency flow, hub device-control interactions, and music page behavior.

Firebase Authentication, Firestore, realtime snapshots, navigation, and browser AudioContext were mocked so the tests can run safely without modifying real backend data.

The final test result is:

- 9 test suites passed
- 49 tests passed
- 0 failed tests

Istanbul coverage reporting was also generated through Jest to visualize tested and untested areas of the web application.

Test Cases

TC1 – Login with valid credentials

Expected result: User is redirected to the hub page.

TC2 – Login with empty email

Expected result: Error message “Email is required.”

TC3 – Login with empty password

Expected result: Error message “Password is required.”

TC4 – Login with invalid credentials

Expected result: User remains on login page and receives an authentication error message.

TC5 – Signup with valid input

Expected result: Account is created and user is redirected to the hub page.

TC6 – Signup with short password

Expected result: Error message that password must be at least 6 characters.

TC7 – Signup with mismatched passwords

Expected result: Error message “Passwords do not match.”

TC8 – Open guest hub from landing page

Expected result: Guest hub page opens correctly.

TC9 – Load hub with existing Firestore data

Expected result: Device states and telemetry values are displayed correctly.

TC10 – Toggle white light

Expected result: Firestore field white_light.state changes between on and off.

TC11 – Toggle door

Expected result: Firestore field door.state changes between open and closed.

TC12 – Toggle window

Expected result: Firestore field window.state changes between open and closed.

TC13 – Toggle fan INA / fan INB

Expected result: Corresponding Firestore fields update correctly.

TC14 – External backend update while hub is open

Expected result: Hub values refresh automatically through snapshot synchronization.

TC15 – Missing or incomplete Firestore data

Expected result: The page should not crash; missing values should be handled safely where possible.

TC16 – Automated login page rendering test

Expected result: Login page renders the title, subtitle, email input, password input, and sign-in button.

TC17 – Automated login validation test

Expected result: Empty email and empty password inputs show the correct validation messages.

TC18 – Automated successful login test

Expected result: Firebase login function is called with the provided email and password, and the user is redirected to the hub page.

TC19 – Automated failed login test

Expected result: If Firebase rejects the login request, the page displays a user-friendly error message.

TC20 – Automated signup validation test

Expected result: Missing full name, missing email, short password, and mismatched passwords show the correct validation messages.

TC21 – Automated successful signup test

Expected result: Firebase account creation, profile update, Firestore user document creation, and redirect to hub are executed using mocked Firebase functions.

TC22 – Automated Firebase configuration test

Expected result: The application exports Firebase Auth and Firestore configuration objects correctly.

TC23 – Activate voice recognition

Expected result: Microphone activates and transcript updates during speech input.

TC24 – Navigate to another page using voice command

Expected result: Application redirects to the requested page.

TC25 – Toggle device using voice command

Expected result: Corresponding device state updates correctly in Firestore and the UI.

TC26 – Unsupported browser speech test

Expected result: Voice recognition may not function correctly on unsupported browsers; Chrome-based browsers are recommended.

TC27 - Multiple commands at once

Expected result: Corresponding device states update all in order

Deliverables

The output of this sprint testing will be:

- a set of executed manual test cases
- a list of detected issues
- retest notes after corrections
- recommendations for future automated testing

Additional automated testing deliverables:

- Jest configuration for the Next.js web application
- Jest setup file with React Testing Library support
- Automated tests for login and signup screens
- Automated tests for Firebase Auth and Firestore configuration
- Istanbul coverage report generated from Jest

Final testing deliverables:

- Jest setup for the Next.js web application
- React Testing Library setup
- Automated tests for authentication flows
- Automated tests for Firebase configuration
- Automated component tests
- Automated static page tests
- Automated emergency page tests
- Automated hub/device-control tests
- Automated music page tests
- Istanbul coverage report